South Valley National University

Computers and Artificial Intelligence

Level 4 - CS

Computation Theory

CS401

Learning Objectives

The overall goals for a student completing this content are to:

- Understand the fundamental concepts of theoretical computer science, including the basic models of computation and formal language theory.
- Develop the ability to design and analyze simple computational models like finite automata.
- Grasp the relationship between different models of computation and formal language classes, particularly Regular Languages and Context-Free Languages.
- Apply theoretical concepts to practical problems, such as designing systems that recognize patterns or parse simple language structures.

Chapter-Specific Learning Outcomes

Chapter 1: Introduction

Upon successful completion of this chapter, the student will be able to:

- Explain the purpose and motivation for studying the Theory of Computation, including its core areas: Complexity Theory, Computability Theory, and Automata Theory.
- Define and recall the essential mathematical prerequisites needed for the course, such as sets, strings, languages, and proof techniques.

Chapter 2: Regular Languages and Finite Automata

Upon successful completion of this chapter, the student will be able to:

- Describe the structure and operation of a Deterministic Finite Automaton (DFA), including its formal definition (states, alphabet, transition function, start state, accept states).
- Design a DFA to accept a specific Regular Language defined by a set of strings or a language description (e.g., controlling a toll gate).
- Define and apply the regular operations (union, concatenation, star) to languages.

- Describe the structure and operation of a Nondeterministic Finite Automaton (NFA), including its formal definition.
- Convert a given NFA into an equivalent DFA (demonstrating the equivalence of DFAs and NFAs).
- Prove that the class of regular languages is closed under the regular operations.
- Define and write Regular Expressions to describe regular languages.
- Convert a regular expression into an equivalent NFA/DFA and convert a DFA into an equivalent regular expression (demonstrating the equivalence of regular expressions and regular languages).

Chapter 3: Context-Free Languages (Introduction)

Upon successful completion of this chapter, the student will be able to:

- Define a Context-Free Grammar (CFG), including its formal components (variables, terminals, production rules, start variable).
- Construct a CFG to generate simple Context-Free Languages (CFLs), such as properly nested parentheses or languages known to be nonregular.
- Differentiate between Regular Languages and Context-Free Languages through examples and their generating mechanisms (finite automata vs. context-free grammars).

Table of Contents

Learning Objectives I
Chapter-Specific Learning Outcomes I

Chapter 1 Introduction 1
1.1 Purpose and motivation 1
1.1.1 Complexity theory 1
1.1.2 Computability theory 2
1.1.3 Automata theory 2
1.2 Mathematical preliminaries 3
Chapter 2 6
2.1 An example: Controlling a toll gate 6
2.2 Deterministic finite automata 7
2.3 Designing finite automata 10
2.4 Regular operations 12
2.5 Nondeterministic finite automata 13
2.5.1 A first example 13
2.5.2 A second example 16
2.5.3 A third example 17
2.5.4 Definition of nondeterministic finite automaton 18
2.6 Equivalence of DFAs and NFAs 20
2.6.1 An example 23
2.7 Closure under the regular operations 27
2.8 Regular expressions 32
2.9 Equivalence of regular expressions and regular languages 36
2.8.1 Every regular expression describes a regular language 37
2.8.2 Converting a DFA to a regular expression 40
Exercises 45

Chapter 3 Context-Free Languages 49
3.1 Context-free grammars 49
3.2 Examples of context-free grammars 52
1.2.1 Properly nested parentheses 52
1.2.2 A context-free grammar for a nonregular language 53
1.2.3 A context-free grammar for the complement of a nonregular language 54
Reference 56

Chapter 1 Introduction

1.1 Purpose and motivation

This course is on the Theory of Computation, which tries to answer the following questions:

- What are the mathematical properties of computer hardware and software?
- What is a computation and what is an algorithm? Can we give rigorous mathematical definitions of these notions?
- What are the limitations of computers? Can "everything" be computed? (As we will see, the answer to this question is "no".)

Purpose of the Theory of Computation: Develop formal mathematical models of computation that reflect real-world computers.

This field of research was started by mathematicians and logicians in the 1930's, when they were trying to understand the meaning of a "computation". A central question asked was whether all mathematical problems can be solved in a systematic way. The research that started in those days led to computers as we know them today.

Nowadays, the Theory of Computation can be divided into the following three areas: Complexity Theory, Computability Theory, and Automata Theory.

1.1.1 Complexity theory

The main question asked in this area is "What makes some problems computationally hard and other problems easy?"

Informally, a problem is called "easy", if it is efficiently solvable. Examples of "easy" problems are (i) sorting a sequence of, say, 1,000,000 numbers, (ii) searching for a name in a telephone directory, and (iii) computing the fastest way to drive from Ottawa to Miami. On the other hand, a problem is called "hard", if it cannot be solved efficiently, or if we don't know whether it can be solved efficiently. Examples of "hard" problems are (i) time table scheduling for all courses at Carleton, (ii) factoring a 300-digit integer

into its prime factors, and (iii) computing a layout for chips in VLSI.

Central Question in Complexity Theory: Classify problems ac- cording to their degree of "difficulty". Give a rigorous proof that problems that seem to be "hard" are really "hard".

1.1.2 Computability theory

In the 1930's, Gödel, Turing, and Church discovered that some of the fundamental mathematical problems cannot be solved by a "computer". (This may sound strange, because computers were invented only in the 1940's). An example of such a problem is "Is an arbitrary mathematical statement true or false?" To attack such a problem, we need formal definitions of the notions of

- computer,
- algorithm, and
- computation.

The theoretical models that were proposed in order to understand solvable and unsolvable problems led to the development of real computers.

Central Question in Computability Theory: Classify problems as being solvable or unsolvable.

1.1.3 Automata theory

Automata Theory deals with definitions and properties of different types of "computation models". Examples of such models are:

- Finite Automata: These are used in text processing, compilers, and hardware design.
- Context-Free Grammars: These are used to define programming languages and in Artificial Intelligence.

- Turing Machines: These form a simple abstract model of a "real" computer, such as your PC at home.

> Central Question in Automata Theory: Do these models have the same power, or can one model solve more problems than the other?

1.2 Mathematical preliminaries

Throughout this course, we will assume that you know the following mathe- matical concepts:

1. A set is a collection of well-defined objects. Examples are (i) the set of all Dutch Olympic Gold Medallists, (ii) the set of all pubs in Ottawa, and (iii) the set of all even natural numbers.
2. If A and B are sets, then A is a subset of B, written as A $\subseteq$ B, if every element of A is also an element of B. For example, the set of even natural numbers is a subset of the set of all natural numbers. Every set A is a subset of itself, i.e., $A \subseteq A$. The empty set is a subset of every set A, i.e., $\emptyset \subseteq A$.
3. If B is a set, then the power set (B) of B is defined to be the set of all subsets of B:

$$P(B) = \{A : A \subseteq B\}.$$

   Observe that $\emptyset \in P(B)$ and $B \in P(B)$.

4. If A and B are two sets, then

(a) their union is defined as

$$A \cup B = \{x : x \in A \text{ or } x \in B\},$$

(b) their intersection is defined as

$$A \cap B = \{x : x \in A \text{ and } x \in B\},$$

(c) their difference is defined as

$$A \setminus B = \{x : x \in A \text{ and } x \not\in B\},$$

(d) the Cartesian product of A and B is defined as

$$A \times B = \{(x, y) : x \in A \text{ and } y \in B\},$$

(e) the complement of A is defined as

$$A = \{x : x \notin A\}.$$

5. A binary relation on two sets A and B is a subset of $A \times B$.
6. A function f from A to B, denoted by $f : A \to B$, is a binary relation R, having the property that for each element a A, there is exactly one ordered pair in R, whose first component is a. We will also say that f (a) = b, or f maps a to b, or the image of a under f is b. The set A is called the domain of f , and the set $\{b \in B : \text{ there is an } a \in A \text{ with } f(a) = b\}$

   is called the range of f.
7. A function $f : A \to B$ is one-to-one (or injective), if for any two distinct elements a and a' in A, we have $f(a) \neq f(a')$. The function f is onto (or surjective), if for each element $b \in B$, there exists an element $a \in A$, such that $f(a) = b$; in other words, the range of f is equal to the set

   B. A function f is a bijection, if f is both injective and surjective.
8. A binary relation $R \subseteq A \times A$ is an equivalence relation, if it satisfies the following three conditions:

(a) R is reflexive: For every element in $a \in A$, we have $(a, a) \in R$.

(b) R is symmetric: For all a and b in A, if $(a, b) \in R$, then also $(b, a) \in R$.

(c) R is transitive: For all a, b, and c in A, if $(a, b) \in R$ and $(b, c) \in R$, then also $(a, c) \in R$.

9. In the context of strings, an alphabet is a finite set, whose elements are called symbols. Examples of alphabets are $\Sigma = \{0, 1\}$ and $\Sigma = \{a, b, c, \ldots, z\}$.
10. A string over an alphabet Σ is a finite sequence of symbols, where each symbol is an element of Σ. The length of a string w, denoted by $|w|$, is the number of symbols contained in w. The empty string, denoted by ϱ, is the string having length zero. For example, if the alphabet Σ is equal to $\{0, 1\}$, then 10, 1000, 0, 101, and ϱ are strings over Σ, having

    lengths 2, 4, 1, 3, and 0, respectively.
11. A language is a set of strings.

12. The Boolean values are 1 and 0, that represent true and false, respectively. The basic Boolean operations include

(a) negation (or NOT), represented by $\neg$,

(b) conjunction (or AND), represented by $\wedge$,

(c) disjunction (or OR), represented by $\vee$,

(d) exclusive-or (or XOR), represented by $\oplus$,

(e) equivalence, represented by $\leftrightarrow$ or $\Leftrightarrow$,

(f) implication, represented by $\rightarrow$ or $\Rightarrow$.

The following table explains the meanings of these operations.

NOT	AND	OR	XOR	equivalence	implication
$\neg 0 = 1$	$0 \wedge 0 = 0$	$0 \vee 0 = 0$	$0 \oplus 0 = 0$	$0 \leftrightarrow 0 = 1$	$0 \rightarrow 0 = 1$
$\neg 1 = 0$	$0 \wedge 1 = 0$	$0 \vee 1 = 1$	$0 \oplus 1 = 1$	$0 \leftrightarrow 1 = 0$	$0 \rightarrow 1 = 1$
	$1 \wedge 0 = 0$	$1 \vee 0 = 1$	$1 \oplus 0 = 1$	$1 \leftrightarrow 0 = 0$	$1 \rightarrow 0 = 0$
	$1 \wedge 1 = 1$	$1 \vee 1 = 1$	$1 \oplus 1 = 0$	$1 \leftrightarrow 1 = 1$	$1 \rightarrow 1 = 1$

Chapter 2

Finite Automata and Regular Languages

In this chapter, we introduce and analyze the class of languages that are known as regular languages. Informally, these languages can be "processed" by computers having a very small amount of memory.

2.1 An example: Controlling a toll gate

Before we give a formal definition of a finite automaton, we consider an example in which such an automaton shows up in a natural way. We consider the problem of designing a "computer" that controls a toll gate.

When a car arrives at the toll gate, the gate is closed. The gate opens as soon as the driver has payed 25 cents. We assume that we have only three coin denominations: 5, 10, and 25 cents. We also assume that no excess change is returned.

After having arrived at the toll gate, the driver inserts a sequence of coins into the machine. At any moment, the machine has to decide whether or not to open the gate, i.e., whether or not the driver has paid 25 cents (or more). In order to decide this, the machine is in one of the following six states, at any moment during the process:

- The machine is in state q_0, if it has not collected any money yet.
- The machine is in state q_1, if it has collected exactly 5 cents.
- The machine is in state q_2, if it has collected exactly 10 cents.
- The machine is in state q_3, if it has collected exactly 15 cents.
- The machine is in state q_4, if it has collected exactly 20 cents.
- The machine is in state q_5, if it has collected 25 cents or more. Initially (when a car arrives at the toll gate), the machine is in state q_0.

Assume, for example, that the driver presents the sequence (10,5,5,10) of coins.

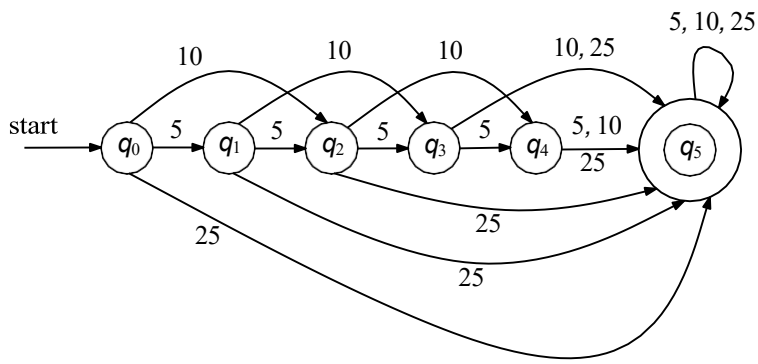
- After receiving the first 10 cents coin, the machine switches from state q_0 to state q_2.

- After receiving the first 5 cents coin, the machine switches from state q_2 to state q_3.

- After receiving the second 5 cents coin, the machine switches from state q_3 to state q_4.

- After receiving the second 10 cents coin, the machine switches from state q_4 to state q_5. At this moment, the gate opens. (Remember that no change is given.)

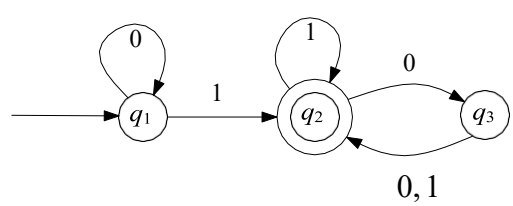
The figure below represents the behavior of the machine for all possible sequences of coins. State q_5 is represented by two circles, because it is a special state: As soon as the machine reaches this state, the gate opens.



Observe that the machine (or computer) only has to remember which state it is in at any given time. Thus, it needs only a very small amount of memory: It has to be able to distinguish between any one of six possible cases and, therefore, it only needs a memory of $\lceil \log 6 \rceil = 3$ bits.

2.2 Deterministic finite automata

Let us look at another example. Consider the following state diagram:



We say that q_1 is the start state and q_2 is an accept state. Consider the input string 1101. This string is processed in the following way:

- Initially, the machine is in the start state q_1.
- After having read the first 1, the machine switches from state q_1 to state q_2.
- After having read the second 1, the machine switches from state q_2 to state q_2. (So actually, it does not switch.)
- After having read the first 0, the machine switches from state q_2 to state q_3.
- After having read the third 1, the machine switches from state q_3 to state q_2.

After the entire string 1101 has been processed, the machine is in state q_2, which is an accept state. We say that string 1101 is accepted by the machine.

Experimenting with this machine on a variety of input strings reveals that it accepts the strings 1, 01, 11, and 0101010101. In fact, M1 accepts any string that ends with a 1, as it goes to it is accept state q2 whenever it reads the symbol 1. In addition, it accepts strings 100, 0100, 110000, and 0101000000, and any string that ends with an even number of 0s following the last 1. It rejects other strings, such as 0, 10, 101000. Can you describe the language consisting of all strings that M1 accepts? We will do so shortly.

- Every binary string having the property that there are an even number of 0s following the rightmost 1, is accepted by this machine.
- Every other binary string is rejected by the machine. Observe that each such string is either empty, consists of 0s only, or has an odd number of 0s following the rightmost 1.

We now come to the formal definition of a finite automaton:

Definition 2.2.1 A finite automaton is a 5-tuple $M = (Q, \Sigma, \delta, q, F)$, where

1. Q is a finite set, whose elements are called states,

2. Σ is a finite set, called the alphabet; the elements of Σ are called symbols,

3. $\delta : Q \times \Sigma \to Q$ is a function, called the transition function,

4. q is an element of Q; it is called the start state,

5. F is a subset of Q; the elements of F are called accept states.

The example given in the beginning of this section is also a finite automaton. For this example, we have $Q = \{q_1, q_2, q_3\}$, $\Sigma = \{0, 1\}$, the start state is q_1, $F = \{q_2\}$, and δ is given by the following table:

	0	1
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2,

Let us denote this finite automaton by M. The language of M, denoted by L(M), is the set of all binary strings that are accepted by M. As we have seen before, we have

L(M) = {w : w contains at least one 1 and ends with an even number of 0s}. We now give a formal definition of the language of a finite automaton:

Definition 2.2.2 Let $M = (Q, \Sigma, \delta, q, F)$ be a finite automaton and let $w = w_1 w_2 \ldots w_n$ be a string over Σ. Define the sequence $r_0, r_1, \ldots, r_n$ of states, in the following way:

- $r_0 = q$,
- $r_{i+1} = \delta(r_i, w_{i+1})$, for $i = 0, 1, \ldots, n-1$.

1. If $r_n \in F$, then we say that M accepts w.

2. If $r_n \notin F$, then we say that M rejects w.

In this definition, w may be the empty string, which we denote by ϱ, and whose length is zero; thus in the definition above, $n = 0$. In this case, the sequence $r_0, r_1, \ldots, r_n$ of

states has length one; it consists of just the state $r_0 = q$. The empty string is accepted by M if and only if the start state q belongs to F .

Definition 2.2.3 Let $M = (Q, \Sigma, \delta, q, F)$ be a finite automaton. The language L(M) accepted by M is defined to be the set of all strings that are accepted by M :

$L(M) = \{w : w \text{ is a string over } \Sigma \text{ and } M \text{ accepts } w\}$.

Definition 2.2.4 A language A is called regular, if there exists a finite automaton M such that $A = L(M)$.

2.3 Designing finite automata

Let's design a finite automaton using the "reader as automaton" method just described. Suppose that you are given some language and want to design a finite automaton that recognizes it.

- First, decide what information to remember about the string as you read it.
- You cannot remember everything you have seen because a finite automaton has a limited number of states, meaning finite memory.
- Imagine the input is extremely long (for example, "from here to the moon"); it's impossible to remember the entire string.
- The automaton's memory is limited—like having a single sheet of paper with small storage capacity.
- Luckily, for many languages, you don't need to remember the whole input.
- You only need to keep certain crucial information, which depends on the specific language being processed.

Example:

- Alphabet: {0,1}
- Language: all strings that contain an odd number of 1s
- Goal: construct a finite automaton E1 to recognize this language.
- When simulating the automaton, you read the input symbol by symbol (0s and 1s).
- Question: do you need to remember the entire string to know whether the number

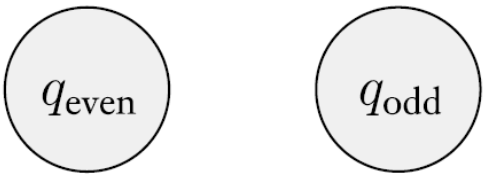
of 1s is odd?

- No. You just need to remember whether the number of 1s so far is even or odd.

- As you read each new symbol:
  - If the symbol is 1, flip the answer (even → odd, or odd → even).
  - If the symbol is 0, keep the current answer unchanged.

Designing E1:

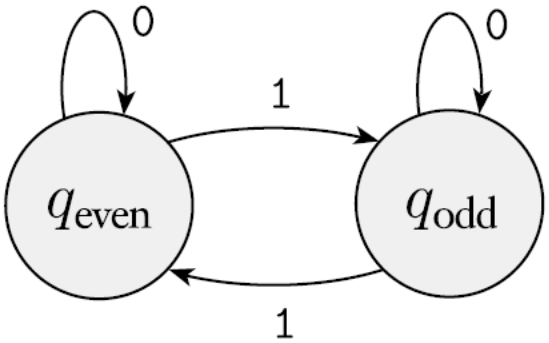
- Identify what information must be remembered during reading.
- Represent this information as a finite list of possibilities:
  1. Even so far
  2. Odd so far



- Assign a state to each possibility — these become the states of E1.

Transitions:

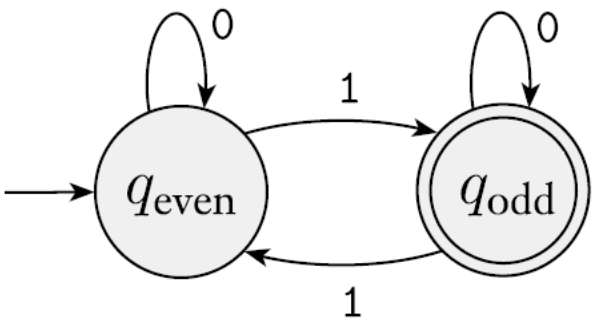
- Define how to move between states when reading symbols:
  - Let q_even represent "even so far."
  - Let q_odd represent "odd so far."
  - On reading 1, flip the state (q_even ↔ q_odd).
  - On reading 0, stay in the same state.



Next, you set the start state to be the state corresponding to the possibility associated

with having seen 0 symbols so far (the empty string ε).

- In this case, the start state corresponds to state q_even because 0 is an even number.
- Last, set the accept states to be those corresponding to possibilities where you want to accept the input string.
- Set q_odd to be an accept state because you want to accept when you have seen an odd number of 1s.
- These additions are shown in the following figure.



Example

A = {w ∈ {0, 1}∗ : w has a 1 in the third position from the right}

2.4 Regular operations

In this section, we define three operations on languages. Later, we will answer the question whether the set of all regular languages is closed under these operations. Let A and B be two languages over the same alphabet.

1. The union of A and B is defined as

$$A \cup B = \{w : \ w \in A \text{ or } w \in B\}.$$

2. The concatenation of A and B is defined as

$$AB = \{ww' : \ w \in A \text{ and } w' \in B\}.$$

In words, AB is the set of all strings obtained by taking an arbitrary string w in A and an arbitrary string w′ in B, and gluing them together (such that w is to the left of w′).

3. The star of A is defined as

$$A^* = \{u_1u_2 \ldots u_k : \ k \geq 0 \text{ and } u_i \in A \text{ for all } i = 1, 2, \ldots, k\}.$$

In words, A^* is obtained by taking any finite number of strings in A, and gluing them together. Observe that $k = 0$ is allowed; this corresponds to the empty string ε. Thus, $\varepsilon \in A^*$.

To give an example, let $A = \{0, 01\}$ and $B = \{1, 10\}$. Then

$$A \cup B = \{0, 01, 1, 10\},$$

$$AB = \{01, 010, 011, 0110\},$$

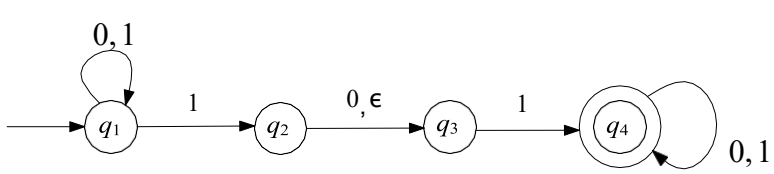
$$A^* = \{\varepsilon, 0, 01, 00, 001, 010, 0101, 000, 0001, 00101, \ldots\}.$$

2.5 Nondeterministic finite automata

We start by giving three examples of nondeterministic finite automata. These examples will show the difference between this type of automata and the deterministic versions that we have considered in the previous sections. After these examples, we will give a formal definition of a nondeterministic finite automaton.

2.5.1 A first example

Consider the following state diagram:



You will notice three differences with the finite automata that we have seen until now. First, if the automaton is in state q_1 and reads the symbol 1, then it has two options: Either it stays in state q_1, or it switches to state q_2. Second, if the automaton is in state q_2, then it can switch to state q_3 without reading a symbol ; this is indicated by the edge having the empty string ϱ as label. Third, if the automaton is in state q_3 and reads the symbol 0, then it cannot continue.

Let us see what this automaton can do when it gets the string 010110 as input. Initially, the automaton is in the start state q_1.

- Since the first symbol in the input string is 0, the automaton stays in state q_1 after having read this symbol.

- The second symbol is 1, and the automaton can either stay in state q_1 or switch to state q_2.

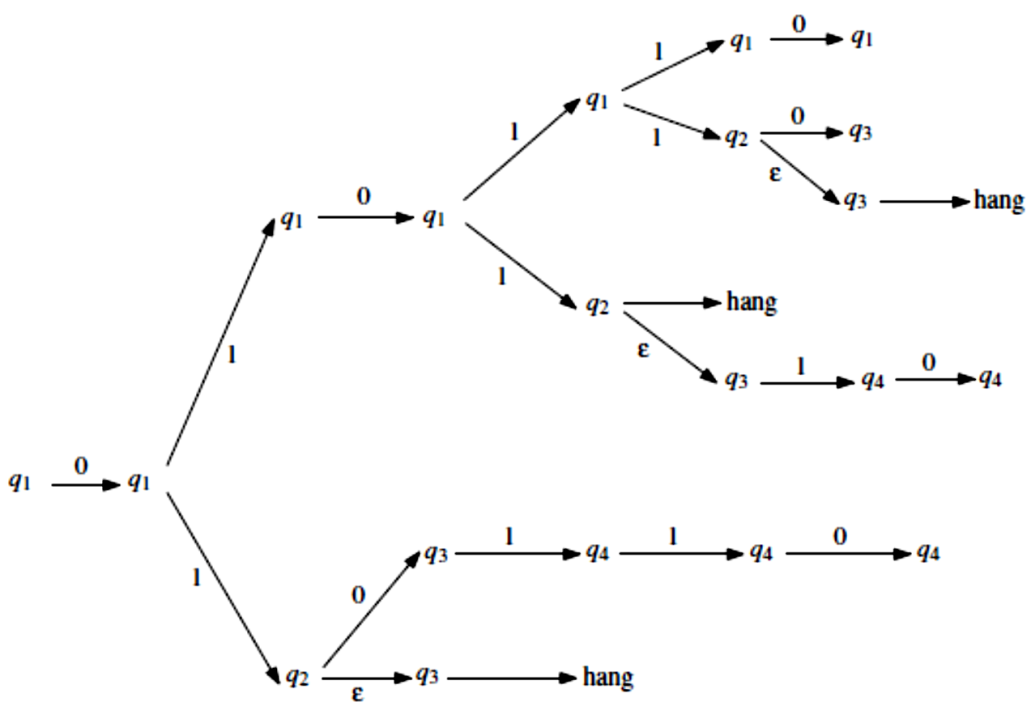
– If the automaton stays in state q_1, then it is still in this state after having read the third symbol.

– If the automaton switches to state q_2, then it again has two options:

∗ Either read the third symbol in the input string, which is 0, and switch to state q_3,

∗ or switch to state q_3, without reading the third symbol.

If we continue in this way, then we see that, for the input string 010110, there are seven possible computations. All these computations are given in the figure below.



Consider the lowest path in the figure above:

- When reading the first symbol, the automaton stays in state q_1.
- When reading the second symbol, the automaton switches to state q_2.
- The automaton does not read the third symbol (equivalently, it "reads" the empty string ϱ), and switches to state q_3. At this moment, the automaton cannot continue: The third symbol is 0, but there is no edge leaving q_3 that is labeled 0, and there is no edge leaving q_3 that is labeled ϱ. Therefore, the computation hangs at this point.

From the figure, you can see that, out of the seven possible computations, exactly two end in the accept state q_4 (after the entire input string 010110 has been read). We say that the automaton accepts the string 010110, because there is at least one computation that ends in the accept state.

Now consider the input string 010. In this case, there are three possible computations:

1. $q_1 \xrightarrow{0} q_1 \xrightarrow{1} q_1 \xrightarrow{0} q_1$
2. $q_1 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \xrightarrow{0} q_3$
3. $q_1 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \xrightarrow{\epsilon} q_3 \rightarrow \text{hang}$

None of these computations ends in the accept state (after the entire input string 010 has been read). Therefore, we say that the automaton rejects the input string 010.

The state diagram given above is an example of a nondeterministic finite automaton (NFA). Informally, an NFA accepts a string, if there exists at least one path in the state diagram that (i) starts in the start state, (ii) does not hang before the entire string has been read, and (iii) ends in an accept state. A string for which (i), (ii), and (iii) does not hold is rejected by the NFA.

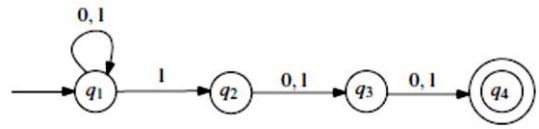
The NFA given above accepts all binary strings that contain 101 or 11 as a substring. All other binary strings are rejected.

2.5.2 A second example

Let A be the language

$A = \{w \in \{0, 1\}^* : w \text{ has a 1 in the third position from the right}\}$.

The following state diagram defines an NFA that accepts all strings that are in A, and rejects all strings that are not in A.



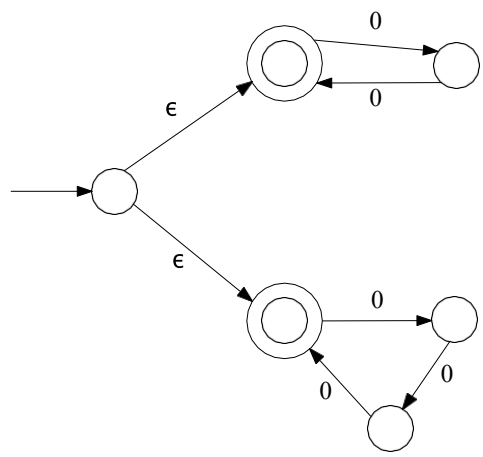
This NFA does the following. If it is in the start state q_1 and reads the symbol 1, then it either stays in state q_1 or it "guesses" that this symbol is the third symbol from the right in the input string. In the latter case, the NFA switches to state q_2, and then it "verifies" that there are indeed exactly two remaining symbols in the input string. If there are more than two remaining symbols, then the NFA hangs (in state q_4) after having read the next two symbols.

Observe how this guessing mechanism is used: The automaton can only read the input string once, from left to right. Hence, it does not know when it reaches the third symbol from the right. When the NFA reads a 1, it can guess that this is the third symbol from the right; after having made this guess, it verifies whether or not the guess was correct.

In Section 2.2.3, we have seen a DFA for the same language A. Observe that the NFA has a much simpler structure than the DFA.

2.5.3 A third example

Consider the following state diagram, which defines an NFA whose alphabet is $\{0\}$.



This NFA accepts the language

$$A = \{0^k : k \equiv 0 \bmod 2 \text{ or } k \equiv 0 \bmod 3\},$$

where 0^k is the string consisting of k many 0s. (If $k = 0$, then $0^k = \varrho$.) Observe that A is the union of the two languages

$$A_1 = \{0^k : k \equiv 0 \bmod 2\}$$

and

$$A_2 = \{0^k : k \equiv 0 \bmod 3\}$$

The NFA basically consists of two DFAs: one of these accepts A_1, whereas the other accepts A_2. Given an input string w, the NFA has to decide whether or not $w \in A$, which is equivalent to deciding whether or not $w \in A_1$ or $w \in A_2$. The NFA makes this decision in the following way: At the start, it "guesses" whether (i) it is going to check whether or not $w \in A_1$ (i.e., the length of w is even), or (ii) it is going to check whether or not $w \in A_2$ (i.e., the length of w is a multiple of 3). After having made the guess, it verifies whether or not the guess was correct. If $w \in A$, then there exists a way of making the correct guess and verifying that w is indeed an element of A (by ending in an accept state). If $w \notin A$, then no matter which guess is made, the NFA will never end in an accept state.

2.5.4 Definition of nondeterministic finite automaton

The previous examples give you an idea what nondeterministic finite au- tomata are and how they work. In this section, we give a formal definition of these automata.
For any alphabet Σ, we define Σ_ϱ to be the set

$$\Sigma_\varrho = \Sigma \cup \{\varrho\}.$$

Recall the notion of a power set : For any set Q, the power set of Q, denoted by P(Q), is the set of all subsets of Q, i.e.,

$$P(Q) = \{R : R \subseteq Q\}.$$

Definition 2.4.1 A nondeterministic finite automaton (NFA) is a 5-tuple $M = (Q, \Sigma, \delta, q, F)$, where

1. Q is a finite set, whose elements are called states,
2. Σ is a finite set, called the alphabet; the elements of Σ are called symbols,
3. $\delta : Q \times \Sigma_\varrho \to P(Q)$ is a function, called the transition function,
4. q is an element of Q; it is called the start state,
5. F is a subset of Q; the elements of F are called accept states.

As for DFAs, the transition function δ can be thought of as the "program" of the finite automaton $M = (Q, \Sigma, \delta, q, F)$:

- Let $r \in Q$, and let $a \in \Sigma_\varrho$. Then $\delta(r, a)$ is a (possibly empty) subset of Q. If the NFA M is in state r, and if it reads a (where a may be the empty string ϱ), then M can switch from state r to any state in $\delta(r, a)$. If $\delta(r, a) = \emptyset$, then M cannot continue and the computation hangs.

The example given in Section 2.4.1 is an NFA, where $Q = \{q_1, q_2, q_3, q_4\}$, $\Sigma = 0, 1$, the start state is q_1, the set of accept states is $F = q_4$, and the transition function

δ is given by the following table:

	0	1	ϱ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

Definition 2.4.2 Let $M = (Q, \Sigma, \delta, q, F)$ be an NFA, and let $w \in \Sigma^*$. We say that M accepts w, if w can be written as $w = y_1 y_2 \ldots y_m$, where $y_i \in \Sigma_\varrho$ for all i with $1 \leq i \leq m$, and there exists a sequence $r_0, r_1, \ldots, r_m$ of states in Q, such that

- $r_0 = q$,
- $r_{i+1} \in \delta(r_i, y_{i+1})$, for $i = 0, 1, \ldots, m-1$, and
- $r_m \in F$.

Otherwise, we say that M rejects the string w.

The NFA in the example in Section 2.4.1 accepts the string 01100. This can be seen by taking

- $w = 01\varrho100 = y_1y_2y_3y_4y_5y_6$, and
- $r_0 = q_1$, $r_1 = q_1$, $r_2 = q_2$, $r_3 = q_3$, $r_4 = q_4$, $r_5 = q_4$, and $r_6 = q_4$.

Definition 2.4.3 Let $M = (Q, \Sigma, \delta, q, F)$ be an NFA. The language $L(M)$ accepted by M is defined as

$$L(M) = \{w \in \Sigma^* : M \text{ accepts } w\}.$$

2.6 Equivalence of DFAs and NFAs

You may have the impression that nondeterministic finite automata are more powerful than deterministic finite automata. In this section, we will show that this is not the case. That is, we will prove that a language can be accepted by a DFA if and only if it can be accepted by an NFA. In order to prove this, we will show how to convert an arbitrary NFA to a DFA that accepts the same language.

What about converting a DFA to an NFA? Well, there is (almost) nothing to do, because a DFA is also an NFA. This is not quite true, because

the transition function of a DFA maps a state and a symbol to a state, whereas

the transition function of an NFA maps a state and a symbol to a set of zero or more states.

The formal conversion of a DFA to an NFA is done as follows: Let $M = (Q, \Sigma, \delta, q, F)$ be a DFA. Recall that δ is a function $\delta : Q \times \Sigma \to Q$. We define the function $\delta' : Q \times \Sigma_\varrho \to P(Q)$ as follows. For any $r \in Q$ and for any $a \in \Sigma_\varrho$,

$$\delta'(r, a) = \begin{cases} \{\delta(r, a)\} & \text{if } a \neq \epsilon, \\ \emptyset & \text{if } a = \epsilon. \end{cases}$$

Then $N = (Q, \Sigma, \delta', q, F)$ is an NFA, whose behavior is exactly the same as that of the DFA M ; the easiest way to see this is by observing that the state diagrams of M and N are equal. Therefore, we have $L(M) = L(N)$.

In the rest of this section, we will show how to convert an NFA to a DFA:

Theorem 2.5.1 Let $N = (Q, \Sigma, \delta, q, F)$ be a nondeterministic finite automa- ton. There exists a deterministic finite automaton M, such that $L(M) = L(N)$.

Proof. Recall that the NFA N can (in general) perform more than one computation on a given input string. The idea of the proof is to construct a DFA M that runs all these different computations simultaneously. (We have seen this idea already in the proof of Theorem 2.3.1.) To be more precise, the DFA M will have the following property:

- the state that M is in after having read an initial part of the input string corresponds exactly to the set of all states that N can reach after having read the same part of the input string.

We start by presenting the conversion for the case when N does not contain ϱ-transitions. In other words, the state diagram of N does not contain any edge that has ϱ as a label. (Later, we will extend the conversion to the general case.) Let the DFA M be defined as $M = (Q', \Sigma, \delta', q', F')$, where

- the set Q' of states is equal to $Q' = P(Q)$; observe that $|Q'| = 2^{|Q|}$,

  the start state q' is equal to $q' = q$; so M has the "same" start state as N,

  the set F' of accept states is equal to the set of all elements R of Q' having the property that R contains at least one accept state of N, i.e.,

$$F' = \{R \in Q' : R \cap F \neq \emptyset\},$$

- the transition function $\delta' : Q' \times \Sigma \to Q'$ is defined as follows: For each $R \in Q'$ and for each $a \in \Sigma$,

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a).$$

Let us see what the transition function δ' of M does. First observe that, since N is an NFA, $\delta(r, a)$ is a subset of Q. This implies that $\delta'(R, a)$ is the union of subsets of Q and, therefore, also a subset of Q. Hence, $\delta'(R, a)$ is an element of Q'.

The set $\delta(r, a)$ is equal to the set of all states of the NFA N that can be reached from state r by reading the symbol a. We take the union of these sets $\delta(r, a)$, where r ranges over all elements of R, to obtain the new set $\delta'(R, a)$. This new set is the state that the DFA M reaches from state R, by reading the symbol a.

In this way, we obtain the correspondence that was given in the beginning of this proof.

After this warming-up, we can consider the general case. In other words, from now on, we allow ϱ-transitions in the NFA N. The DFA M is defined as above, except that the start state q' and the transition function δ' have to be modified. Recall that a computation of the NFA N consists of the following:

1. Start in the start state q and make zero or more ϱ-transitions.
2. Read one "real" symbol of Σ and move to a new state (or stay in the current state).
3. Make zero or more ϱ-transitions.
4. Read one "real" symbol of Σ and move to a new state (or stay in the current state).
5. Make zero or more ϱ-transitions.
6. Etc.

The DFA M will simulate this computation in the following way:

- Simulate 1. in one single step. As we will see below, this simulation is implicitly encoded in the definition of the start state q' of M.
- Simulate 2. and 3. in one single step.
- Simulate 4. and 5. in one single step.
- Etc.

Thus, in one step, the DFA M simulates the reading of one "real" symbol of Σ, followed by making zero or more ϱ-transitions.

To formalize this, we need the notion of ϱ-closure. For any state r of the NFA N, the ϱ-closure of r, denoted by $C_\varrho(r)$, is defined to be the set of all states of N that can be reached from r, by making zero or more ϱ-transitions. For any state R of the DFA M

$$C_\epsilon(R) = \bigcup_{r \in R} C_\epsilon(r).$$

(hence, $R \subseteq Q$), we define

How do we define the start state q' of the DFA M? Before the NFA N reads its first "real" symbol of Σ, it makes zero or more ϱ-transitions. In other words, at the moment when N reads the first symbol of Σ, it can be in any state of $C_\varrho(q)$. Therefore, we define q' to be

$$q' = C_\varrho(q) = C_\varrho(\{q\}).$$

How do we define the transition function δ' of the DFA M ? Assume that M is in state R, and reads the symbol a. At this moment, the NFA N would have been in any state r of R. By reading the symbol a, N can switch to any state in $\delta(r, a)$, and then make zero or more ϱ-transitions. Hence, the NFA can switch to any state in the set $C_\varrho(\delta(r, a))$. Based on this, we define

$\delta'(R, a)$ to be

$$\delta'(R, a) = \bigcup_{r \in R} C_\epsilon(\delta(r, a)).$$

To summarize, the NFA $N = (Q, \Sigma, \delta, q, F)$ is converted to the DFA $M = (Q', \Sigma, \delta', q', F')$, where

- $Q' = P(Q)$,
- $q' = C_\varrho(\{q\})$,
- $F' = \{R \in Q' : R \cap F \neq \emptyset\}$,
- $\delta' : Q' \times \Sigma \to Q'$ is defined as follows: For each $R \in Q'$ and for each $a \in \Sigma$,

$$\delta'(R, a) = \bigcup_{r \in R} C_\epsilon(\delta(r, a)).$$

The results proved until now can be summarized in the following theorem.

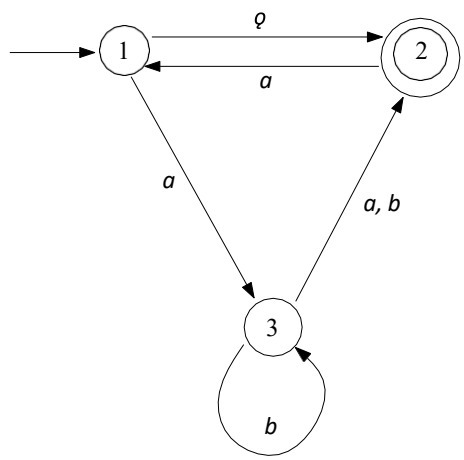
Theorem 2.5.2 Let A be a language. Then A is regular if and only if there exists a nondeterministic finite automaton that accepts A.

2.6.1 An example

Consider the NFA $N = (Q, \Sigma, \delta, q, F)$, where $Q = \{1, 2, 3\}$, $\Sigma = \{a, b\}$, $q = 1$, $F = \{2\}$, and δ is given by the following table:

	a	b	ϱ
1	{3}	∅	{2}
2	{1}	∅	∅
3	{2}	{2, 3}	∅

The state diagram of N is as follows:



We will show how to convert this NFA N to a DFA M that accepts the same language. Following the proof of Theorem 2.5.1, the DFA M is specified by $M = (Q', \Sigma, \delta', q', F')$, where each of the components is defined below.

- $Q' = P(Q)$. Hence,

$$Q' = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}.$$

$q' = C_\varrho(\{q\})$. Hence, the start state q' of M is the set of all states of N that can be reached from N's start state $q = 1$, by making zero or more ϱ-transitions. We obtain

$$q' = C_\varrho(\{q\}) = C_\varrho(\{1\}) = \{1, 2\}.$$

- $F' = \{R \in Q' : R \cap F \neq \emptyset\}$. Hence, the accept states of M are those states that contain the accept state 2 of N. We obtain

$$F' = \{\{2\}, \{1, 2\}, \{2, 3\}, \{1, 2, 3\}\}.$$

- $\delta' : Q' \times \Sigma \to Q'$ is defined as follows: For each $R \in Q'$ and for each

a ∈ Σ,

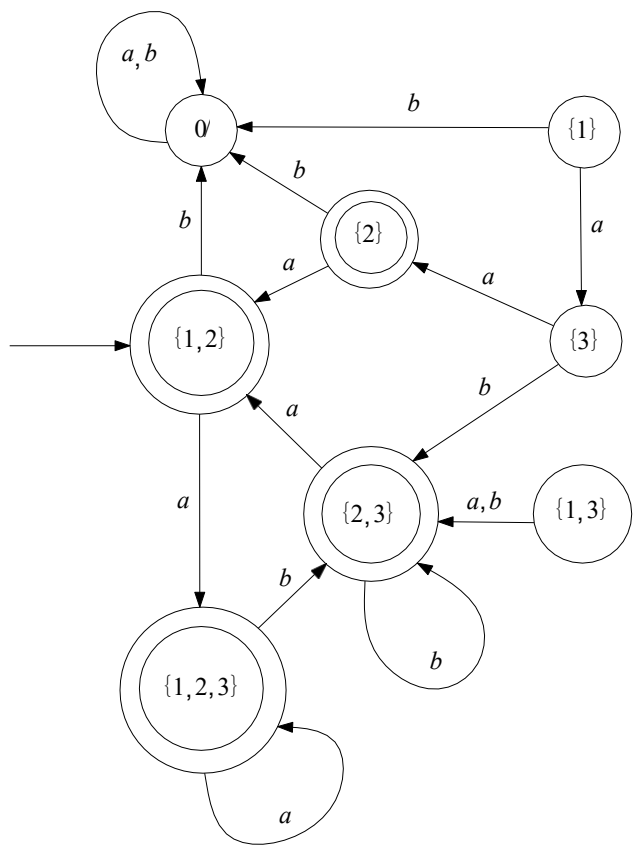
$$\delta'(R, a) = \bigcup_{r \in R} C_\epsilon(\delta(r, a)).$$

In this example δ′ is given by

$\delta'(\emptyset, a) = \emptyset$	$\delta'(\emptyset, b) = \emptyset$
$\delta'(\{1\}, a) = \{3\}$	$\delta'(\{1\}, b) = \emptyset$
$\delta'(\{2\}, a) = \{1, 2\}$	$\delta'(\{2\}, b) = \emptyset$
$\delta'(\{3\}, a) = \{2\}$	$\delta'(\{3\}, b) = \{2, 3\}$
$\delta'(\{1, 2\}, a) = \{1, 2, 3\}$	$\delta'(\{1, 2\}, b) = \emptyset$
$\delta'(\{1, 3\}, a) = \{2, 3\}$	$\delta'(\{1, 3\}, b) = \{2, 3\}$
$\delta'(\{2, 3\}, a) = \{1, 2\}$	$\delta'(\{2, 3\}, b) = \{2, 3\}$

$\delta'(\{1, 2, 3\}, a) = \{1, 2, 3\}$ $\delta'(\{1, 2, 3\}, b) = \{2, 3\}$

The state diagram of the DFA M is as follows:



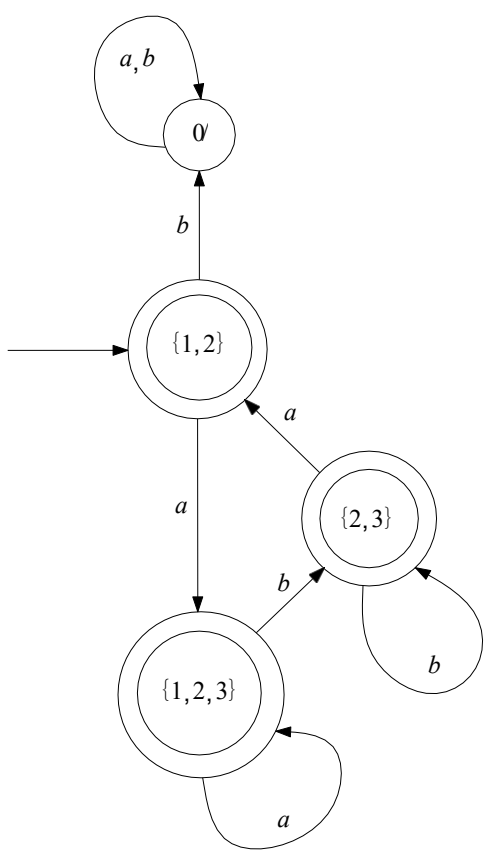
We make the following observations:

- The states $\{1\}$ and $\{1, 3\}$ do not have incoming edges. Therefore, these two states cannot be reached from the start state $\{1, 2\}$.

  The state 3 has only one incoming edge; it comes from the state 1 . Since 1 cannot be reached from the start state, 3 cannot be reached from the start state.

- The state $\{2\}$ has only one incoming edge; it comes from the state 3 . Since 3 cannot be reached from the start state, 2 cannot be reached from the start state.

Hence, we can remove the four states $\{1\}$, $\{2\}$, $\{3\}$, and $\{1, 3\}$. The resulting DFA accepts the same language as the DFA above. This leads to the following state diagram, which depicts a DFA that accepts the same language as the NFA N:



2.7 Closure under the regular operations

In Section 2.3, we have defined the regular operations union, concatenation, and star. We proved in Theorem 2.3.1 that the union of two regular lan- guages is a regular language. We also explained why it is not clear that the concatenation of two regular languages is regular, and that the star of a reg- ular language is regular. In this section, we will see that the concept of NFA, together with Theorem 2.5.2, can be used to give a simple proof of the fact that the regular languages are indeed closed under the regular operations. We start by giving an alternative proof of Theorem 2.3.1:

Theorem 2.6.1 The set of regular languages is closed under the union op- eration, i.e., if A_1 and A_2 are regular languages over the same alphabet Σ, then $A_1 \cup A_2$ is also a regular language.

Proof. Since A_1 is regular, there is, by Theorem 2.5.2, an NFA $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$, such that $A_1 = L(M_1)$. Similarly, there is an NFA $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$, such that $A_2 = L(M_2)$. We may assume that $Q_1 \cap Q_2 = \emptyset$, because otherwise, we can give new "names" to the states of Q_1 and Q_2. From these two NFAs, we will construct an NFA $M = (Q, \Sigma, \delta, q_0, F)$, such that $L(M) = A_1 \cup A_2$. The construction is illustrated in Figure 2.1. The NFA M is defined as follows:

1. $Q = \{q_0\} \cup Q_1 \cup Q_2$, where q_0 is a new state.
2. q_0 is the start state of M.
3. $F = F_1 \cup F_2$.

4. $\delta : Q \times \Sigma_Q \to P(Q)$ is defined as follows: For any $r \in Q$ and for any $a \in \Sigma_Q$,

$$\delta(r,a) = \begin{cases} \delta_1(r,a) & \text{if } r \in Q_1, \\ \delta_2(r,a) & \text{if } r \in Q_2, \\ \{q_1, q_2\} & \text{if } r = q_0 \text{ and } a = \epsilon, \\ \emptyset & \text{if } r = q_0 \text{ and } a \neq \epsilon. \end{cases}$$

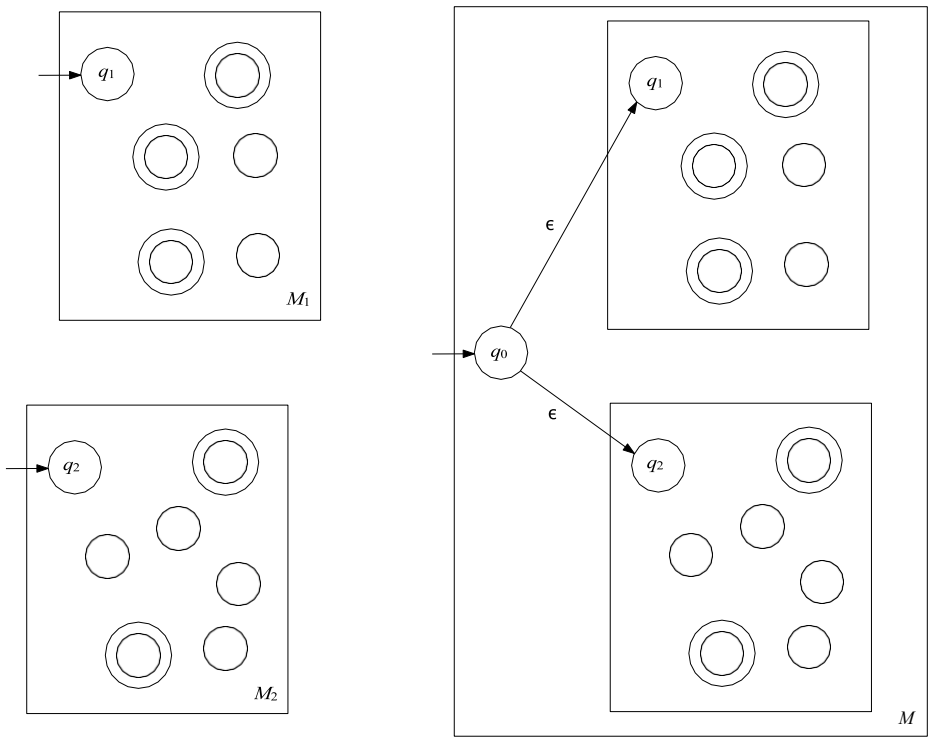


Figure 2.1: The NFA M accepts $L(M_1) \cup L(M_2)$.

Theorem 2.6.2 The set of regular languages is closed under the concatenation operation, i.e., if A_1 and A_2 are regular languages over the same alphabet Σ, then A_1A_2 is also a regular language.

Proof. Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ be an NFA, such that $A_1 = L(M_1)$. Similarly, let $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$ be an NFA, such that $A_2 = L(M_2)$. As in the proof of Theorem 2.6.1, we may assume that $Q_1 \cap Q_2 = \emptyset$. We will construct an NFA $M = (Q, \Sigma, \delta, q_0, F)$, such that $L(M) = A_1A_2$. The construction is illustrated in Figure 2.2.

The NFA M is defined as follows:

1. $Q = Q_1 \cup Q_2$.
2. $q_0 = q_1$.
3. $F = F_2$.

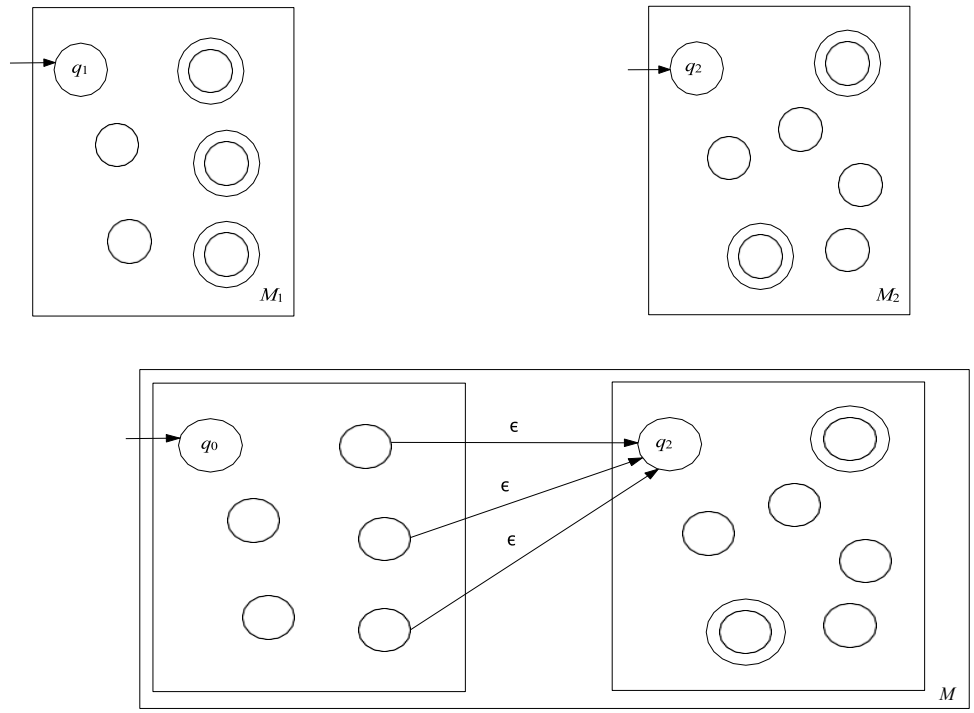


Figure 2.2: The NFA M accepts $L(M_1)L(M_2)$.

---

4. $\delta : Q \times \Sigma_Q \to P(Q)$ is defined as follows: For any $r \in Q$ and for any $a \in \Sigma_Q$,

$$\delta(r,a) = \begin{cases} \delta_1(r,a) & \text{if } r \in Q_1 \text{ and } r \notin F_1, \\ \delta_1(r,a) & \text{if } r \in F_1 \text{ and } a \neq \epsilon, \\ \delta_1(r,a) \cup \{q_2\} & \text{if } r \in F_1 \text{ and } a = \epsilon, \\ \delta_2(r,a) & \text{if } r \in Q_2. \end{cases}$$

Theorem 2.6.3 The set of regular languages is closed under the star oper- ation, i.e., if A is a regular language, then A^* is also a regular language.

Proof. Let Σ be the alphabet of A and let $N = (Q_1, \Sigma, \delta_1, q_1, F_1)$ be an NFA, such that $A = L(N)$. We will construct an NFA $M = (Q, \Sigma, \delta, q_0, F)$, such that $L(M) = A^*$. The construction is illustrated in Figure 2.3. The NFA M is defined as follows:

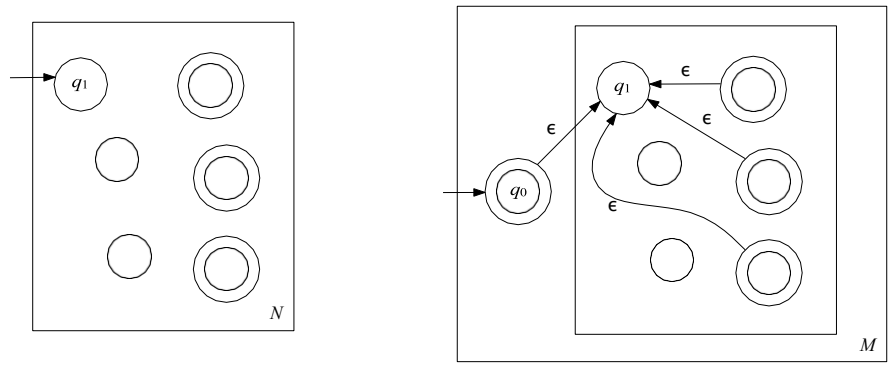


Figure 2.3: The NFA M accepts $(L(N))^*$.

---

1. $Q = \{q_0\} \cup Q_1$, where q_0 is a new state.

2. q_0 is the start state of M.

3. $F = \{q_0\} \cup F_1$. (Since $\varrho \in A^*$, q_0 has to be an accept state.)

4. $\delta : Q \times \Sigma_\varrho \to P(Q)$ is defined as follows: For any $r \in Q$ and for any $a \in \Sigma_\varrho$,

$$\delta(r,a) = \begin{cases} \delta_1(r,a) & \text{if } r \in Q_1 \text{ and } r \notin F_1, \\ \delta_1(r,a) & \text{if } r \in F_1 \text{ and } a \neq \epsilon, \\ \delta_1(r,a) \cup \{q_1\} & \text{if } r \in F_1 \text{ and } a = \epsilon, \\ \{q_1\} & \text{if } r = q_0 \text{ and } a = \epsilon, \\ \emptyset & \text{if } r = q_0 \text{ and } a \neq \epsilon. \end{cases}$$

In the final theorem of this section, we mention (without proof) two more closure properties of the regular languages:

Theorem 2.6.4 The set of regular languages is closed under the complement and intersection operations:

1. If A is a regular language over the alphabet Σ, then the complement

$\overline{A} = \{w \in \Sigma^* : w \notin A\}$

is also a regular language.

2. If A_1 and A_2 are regular languages over the same alphabet Σ, then the intersection

$$A_1 \cap A_2 = \{w \in \Sigma^* : w \in A_1 \text{ and } w \in A_2\}$$

is also a regular language.

2.8 Regular expressions

In this section, we present regular expressions, which are a means to describe languages. As we will see, the class of languages that can be described by regular expressions coincides with the class of regular languages.

Before formally defining the notion of a regular expression, we give some examples. Consider the expression

$$(0 \cup 1)01^*.$$

The language described by this expression is the set of all binary strings

1. that start with either 0 or 1 (this is indicated by $(0 \cup 1)$),
2. for which the second symbol is 0 (this is indicated by 0), and
3. that end with zero or more 1s (this is indicated by 1^*). That is, the language described by this expression is

$$\{00, 001, 0011, 00111, \ldots, 10, 101, 1011, 10111, \ldots\}.$$

Here are some more examples (in all cases, the alphabet is $\{0, 1\}$):

The language $\{w : w \text{ contains exactly two 0s}\}$ is described by the expression

$$1^*01^*01^*.$$

- The language $\{w : w \text{ contains at least two 0s}\}$ is described by the expression

$$(0 \cup 1)^*0(0 \cup 1)^*0(0 \cup 1)^*.$$

- The language $\{w : 1011 \text{ is a substring of } w\}$ is described by the ex- pression

$$(0 \cup 1)^*1011(0 \cup 1)^*.$$

The language $\{ w : \text{the length of } w \text{ is even}\}$ is described by the expression

$$((0 \cup 1)(0 \cup 1))^* .$$

The language $\{ w : \text{the length of } w \text{ is odd}\}$ is described by the expression

$$(0 \cup 1)\,((0 \cup 1)(0 \cup 1))^* .$$

- The language $\{1011, 0\}$ is described by the expression

$$1011 \cup 0.$$

- The language $\{w : \text{the first and last symbols of } w \text{ are equal}\}$ is de- scribed by the expression

$$0(0 \cup 1)^*0 \cup 1(0 \cup 1)^*1 \cup 0 \cup 1.$$

After these examples, we give a formal (and inductive) definition of regular expressions:

Definition 2.7.1 Let Σ be a non-empty alphabet.

1. ϱ is a regular expression.
2. $\emptyset$ is a regular expression.
3. For each $a \in \Sigma$, a is a regular expression.
4. If R_1 and R_2 are regular expressions, then $R_1 \cup R_2$ is a regular expres- sion.
5. If R_1 and R_2 are regular expressions, then R_1R_2 is a regular expression.
6. If R is a regular expression, then R^* is a regular expression.

You can regard 1., 2., and 3. as being the "building blocks" of regular expressions. Items 4., 5., and 6. give rules that can be used to combine regular expressions into new (and "larger") regular expressions. To give an example, we claim that

$(0 \cup 1)^*101(0 \cup 1)^*$

is a regular expression (where the alphabet Σ is equal to $\{0, 1\}$). In order to prove this, we have to show that this expression can be "built" using the "rules" given in Definition 2.7.1. Here we go:

- By 3., 0 is a regular expression.
- By 3., 1 is a regular expression.
- Since 0 and 1 are regular expressions, by 4., $0 \cup 1$ is a regular expression.
- Since $0 \cup 1$ is a regular expression, by 6., $(0 \cup 1)^*$ is a regular expression.
- Since 1 and 0 are regular expressions, by 5., 10 is a regular expression.
- Since 10 and 1 are regular expressions, by 5., 101 is a regular expression.
- Since $(0 \cup 1)^*$ and 101 are regular expressions, by 5., $(0 \cup 1)^*101$ is a regular expression.
- Since $(0 \cup 1)^*101$ and $(0 \cup 1)^*$ are regular expressions, by 5., $(0 \cup 1)^*101(0 \cup 1)^*$ is a regular expression.

Next we define the language that is described by a regular expression:

Definition 2.7.2 Let Σ be a non-empty alphabet.

1. The regular expression $\varnothing$ describes the language $\{\varnothing\}$.
2. The regular expression $\emptyset$ describes the language $\emptyset$.
3. For each $a \in \Sigma$, the regular expression a describes the language $\{a\}$.
4. Let R_1 and R_2 be regular expressions and let L_1 and L_2 be the lan- guages described by them, respectively. The regular expression $R_1 \cup R_2$ describes the language $L_1 \cup L_2$.
5. Let R_1 and R_2 be regular expressions and let L_1 and L_2 be the languages described by them, respectively. The regular expression R_1R_2 describes the language L_1L_2.
6. Let R be a regular expression and let L be the language described by it. The regular expression R^* describes the language L^*.

We consider some examples:

- The regular expression $(0 \cup \varnothing)(1 \cup \varnothing)$ describes the language $\{01, 0, 1, \varnothing\}$.

- The regular expression $0 \cup \varrho$ describes the language $\{0, \varrho\}$, whereas the regular expression 1^* describes the language $\{\varrho, 1, 11, 111, \ldots\}$. There- fore, the regular expression $(0 \cup \varrho)1^*$ describes the language

  $\{0, 01, 011, 0111, \ldots, \varrho, 1, 11, 111, \ldots\}$.

  Observe that this language is also described by the regular expression $01^* \cup 1^*$.

- The regular expression $1^*\emptyset$ describes the empty language, i.e., the lan- guage $\emptyset$. (You should convince yourself that this is correct.)

- The regular expression $\emptyset^*$ describes the language $\{\varrho\}$.

Definition 2.7.3 Let R_1 and R_2 be regular expressions and let L_1 and L_2 be the languages described by them, respectively. If $L_1 = L_2$ (i.e., R_1 and R_2 describe the same language), then we will write $R_1 = R_2$.

Hence, even though $(0 \cup \varrho)1^*$ and $01^* \cup 1^*$ are different regular expressions, we write $(0 \cup \varrho)1^* = 01^* \cup 1^*$,

because they describe the same language.

In Section 2.8.2, we will show that every regular language can be described by a regular expression. The proof of this fact is purely algebraic and uses the following algebraic identities involving regular expressions.

Theorem 2.7.4 Let R_1, R_2, and R_3 be regular expressions. The following identities hold:

1. $R_1\emptyset = \emptyset R_1 = \emptyset$.
2. $R_1\varrho = \varrho R_1 = R_1$.
3. $R_1 \cup \emptyset = \emptyset \cup R_1 = R_1$.
4. $R_1 \cup R_1 = R_1$.

5. $R_1 \cup R_2 = R_2 \cup R_1$.

6. $R_1(R_2 \cup R_3) = R_1R_2 \cup R_1R_3$.

7. $(R_1 \cup R_2)R_3 = R_1R_3 \cup R_2R_3$.

8. $R_1(R_2R_3) = (R_1R_2)R_3$.

9. $\emptyset^* = \varrho$.

10. $\varrho^* = \varrho$.

11. $(\varrho \cup R_1)^* = R_1^*$.

12. $(\varrho \cup R_1)(\varrho \cup R_1)^* = R_1^*$.

13. $R_1^*(\varrho \cup R_1) = (\varrho \cup R_1)R_1^* = R_1^*$.

14. $R_1^*R_2 \cup R_2 = R_1^*R_2$.

15. $R_1(R_2R_1)^* = (R_1R_2)^*R_1$.

16. $(R_1 \cup R_2)^* = (R_1^*R_2)^*R_1^* = (R_2^*R_1)^*R_2^*$.

We will not present the (boring) proofs of these identities, but urge you to convince yourself informally that they make perfect sense. To give an example, we mentioned above that

$$(0 \cup \varrho)1^* = 01^* \cup 1^*.$$

We can verify this identity in the following way:

$$\begin{aligned} (0 \cup \varrho)1^* &= 01^* \cup \varrho 1^* && \text{(by identity 7)} \\ &= 01^* \cup 1^* && \text{(by identity 2)} \end{aligned}$$

2.9 Equivalence of regular expressions and regular languages

In the beginning of Section 2.7, we mentioned the following result:

Theorem 2.8.1 Let L be a language. Then L is regular if and only if there exists a regular expression that describes L.

The proof of this theorem consists of two parts:
In Section 2.8.1, we will prove that every regular expression describes a regular language.

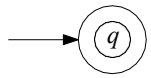
- In Section 2.8.2, we will prove that every DFA M can be converted to a regular expression that describes the language L(M).

These two results will prove Theorem 2.8.1.

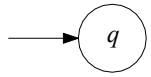
2.8.1 Every regular expression describes a regular language

Let R be an arbitrary regular expression over the alphabet Σ. We will prove that the language described by R is a regular language. The proof is by induction on the structure of R (i.e., by induction on the way R is "built" using the "rules" given in Definition 2.7.1).

The first base case: Assume that R = ϱ. Then R describes the language ϱ . In order to prove that this language is regular, it suffices, by Theorem 2.5.2, to construct an NFA M = (Q, Σ, δ, q, F) that accepts this language. This NFA is obtained by defining Q = {q}, q is the start state, F = q , and δ(q, a) = for all a Σ_{ϱ}. The figure below gives the state diagram of M :



The second base case: Assume that R = $\emptyset$. Then R describes the language $\emptyset$. In order to prove that this language is regular, it suffices, by Theorem 2.5.2, to construct an NFA M = (Q, Σ, δ, q, F) that accepts this language. This NFA is obtained by defining Q = {q}, q is the start state, F = $\emptyset$, and δ(q, a) = $\emptyset$ for all a $\in \Sigma_{\varrho}$. The figure below gives the state diagram of M:



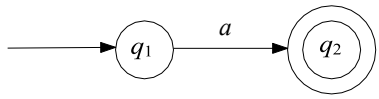
The third base case: Let $a \in \Sigma$ and assume that $R = a$. Then R describes the language $\{a\}$. In order to prove that this language is regular, it suffices, by Theorem 2.5.2, to construct an NFA $M = (Q, \Sigma, \delta, q_1, F)$ that accepts this language. This NFA is obtained by defining $Q = \{q_1, q_2\}$, q_1 is the start state, $F = \{q_2\}$, and

$\delta(q_1, a) = \{q_2\}$,

$\delta(q_1, b) = \emptyset$ for all $b \in \Sigma_Q \setminus \{a\}$,

$\delta(q_2, b) = \emptyset$ for all $b \in \Sigma_Q$.

The figure below gives the state diagram of M:



The first case of the induction step: Assume that $R = R_1 \cup R_2$, where R_1 and R_2 are regular expressions. Let L_1 and L_2 be the languages described by R_1 and R_2, respectively, and assume that L_1 and L_2 are regular. Then R describes the language $L_1 \cup L_2$, which, by Theorem 2.6.1, is regular.

The second case of the induction step: Assume that $R = R_1R_2$, where R_1 and R_2 are regular expressions. Let L_1 and L_2 be the languages described by R_1 and R_2, respectively, and assume that L_1 and L_2 are regular. Then R describes the language L_1L_2, which, by Theorem 2.6.2, is regular.

The third case of the induction step: Assume that $R = (R_1)^*$, where R_1 is a regular expression. Let L_1 be the language described by R_1 and assume that L_1 is regular. Then R describes the language $(L_1)^*$, which, by Theorem 2.6.3, is regular.

This concludes the proof of the claim that every regular expression describes a regular language.

To give an example, consider the regular expression

$$(ab \cup a)^*,$$

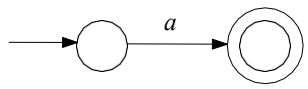
where the alphabet is $\{a, b\}$. We will prove that this regular expression describes a regular language, by constructing an NFA that accepts the language described by this regular expression. Observe how the regular expression is "built":

Take the regular expressions a and b, and combine them into the regular expression ab.

- Take the regular expressions ab and a, and combine them into the regular expression ab ∪ a.
- Take the regular expression ab ∪ a, and transform it into the regular expression (ab ∪ a)*.

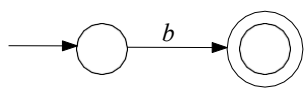
First, we construct an NFA M_1 that accepts the language described by the regular expression a:

M_1



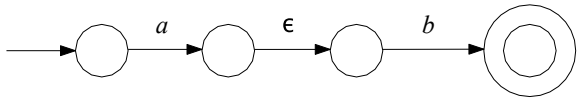
Next, we construct an NFA M_2 that accepts the language described by the regular expression b:

M_2



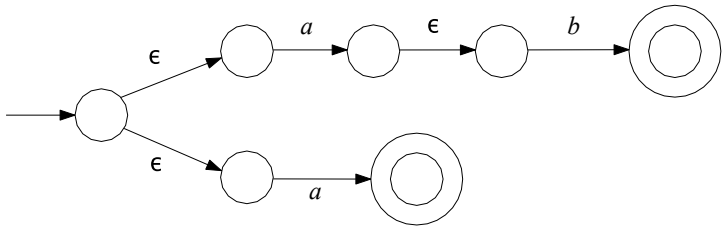
Next, we apply the construction given in the proof of Theorem 2.6.2 to M_1 and M_2. This gives an NFA M_3 that accepts the language described by the regular expression ab:

M_3



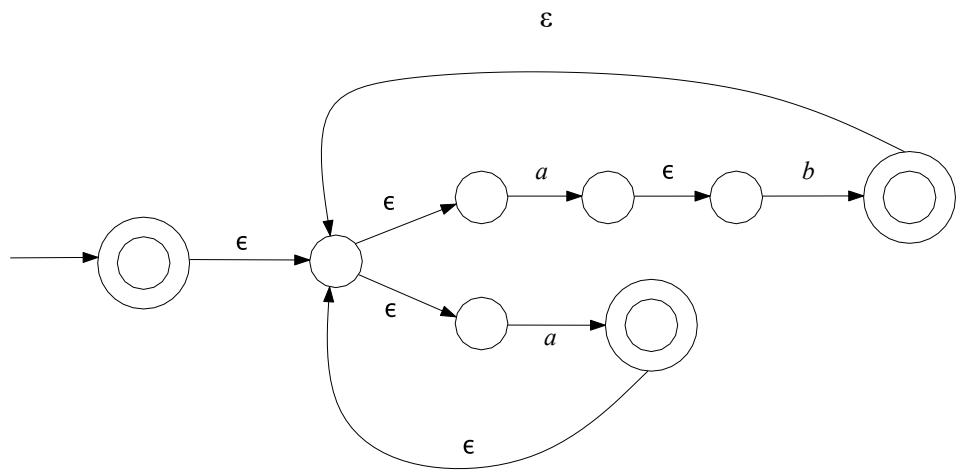
Next, we apply the construction given in the proof of Theorem 2.6.1 to M_3 and M_1. This gives an NFA M_4 that accepts the language described by the regular expression ab ∪ a:

M_4



Finally, we apply the construction given in the proof of Theorem 2.6.3 to M_4. This gives an NFA M_5 that accepts the language described by the regular expression $(ab \cup a)^*$:

M_5



2.8.2 Converting a DFA to a regular expression

In this section, we will prove that every DFA M can be converted to a regular expression that describes the language L(M). In order to prove this result, we need to solve recurrence relations involving languages.

Solving recurrence relations

Let Σ be an alphabet, let B and C be "known" languages in Σ^* such that $\varnothing \notin B$, and let L be an "unknown" language such that

$$L = BL \cup C.$$

Can we "solve" this equation for L? That is, can we express L in terms of B and C?

Consider an arbitrary string u in L. We are going to determine how u looks like. Since $u \in L$ and $L = BL \cup C$, we know that u is a string in $BL \cup C$. Hence, there are two possibilities for u.

1. u is an element of C.

2. u is an element of BL. In this case, there are strings $b \in B$ and $v \in L$ such that $u = bv$.

Since $\varrho \notin B$, we have $b \neq \varrho$ and, therefore, $|v| < |u|$. (Recall that $|v|$ denotes the length, i.e., the number of symbols, of the string v.) Since v is a string in L, which is equal to $BL \cup C$, v is a string in $BL \cup C$. Hence, there are two possibilities for v.

(a) v is an element of C. In this case,

$u = bv$, where $b \in B$ and $v \in C$; thus, $u \in BC$.

(b) v is an element of BL. In this case, there are strings $b' \in B$ and $w \in L$ such that $v = b'w$. Since $\varrho \notin B$, we have $b' \neq \varrho$ and, therefore, $|w| < |v|$. Since w is a string in L, which is equal to $BL \cup C$, w is a string in $BL \cup C$. Hence, there are two possibilities for w.

i. w is an element of C. In this case,

$u = bb'w$, where $b, b' \in B$ and $w \in C$; thus, $u \in BBC$.

ii. w is an element of BL. In this case, there are strings $b'' \in B$ and $x \in L$ such that $w = b''x$. Since $\varrho \notin B$, we have $b'' \neq \varrho$ and, therefore, $|x| < |w|$. Since x is a string in L, which is equal to $BL \cup C$, x is a string in $BL \cup C$. Hence, there are two possibilities for x.

A. x is an element of C. In this case,

$u = bb'b''x$, where $b, b', b'' \in B$ and $x \in C$; thus, $u \in BBBC$.

B. x is an element of BL. Etc., etc.

This process hopefully convinces you that any string u in L can be written as the concatenation of zero or more strings in B, followed by one string in C. In fact, L consists of exactly those strings having this property:

Lemma 2.8.2 Let Σ be an alphabet, and let B, C, and L be languages in Σ^* such that $\varrho \notin B$ and

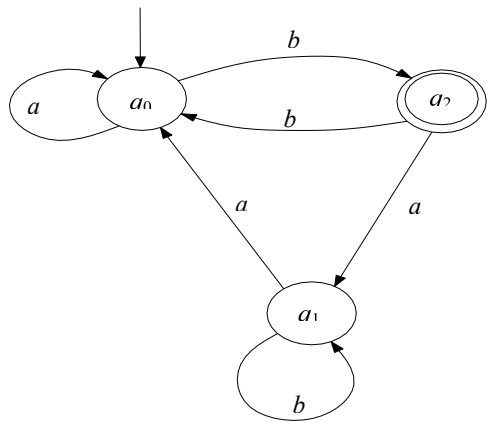
$$L = BL \cup C.$$

Then

$$L = B^*C.$$

An example

Consider the deterministic finite automaton $M = (Q, \Sigma, \delta, q_0, F)$, where $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, q_0 is the start state, $F = \{q_2\}$, and δ is given in the state diagram below. We show how to obtain the regular expression that describes the language accepted by M.



For this case, (2.2) and (2.3) give the following equations:

$$\begin{cases} L_{q_0} = a \cdot L_{q_0} \cup b \cdot L_{q_2} \\ L_{q_1} = a \cdot L_{q_0} \cup b \cdot L_{q_1} \\ L_{q_2} = \epsilon \cup a \cdot L_{q_1} \cup b \cdot L_{q_0} \end{cases}$$

In the third equation, L_{q2} is expressed in terms of L_{q0} and L_{q1}. Hence, if we substitute the third equation into the first one, and use Theorem 2.7.4, then we get

$$\begin{aligned} L_{q0} &= a \cdot L_{q0} \cup b \cdot (\varnothing \cup a \cdot L_{q1} \cup b \cdot L_{q0}) \\ &= (a \cup bb) \cdot L_{q0} \cup ba \cdot L_{q1} \cup b. \end{aligned}$$

We obtain the following set of equations.

$$\begin{cases} L_{q_0} = (a \cup bb) \cdot L_{q_0} \cup ba \cdot L_{q_1} \cup b \\ L_{q_1} = b \cdot L_{q_1} \cup a \cdot L_{q_0} \end{cases}$$

Let $L = L_{q1}$, $B = b$, and $C = a \cdot L_{q0}$. Then $\varrho \notin B$ and the second equation reads $L = BL \cup C$. Hence, by Lemma 2.8.2,

$$L_{q1} = L = B^*C = b^*a \cdot L_{q0} .$$

If we substitute L_{q1} into the first equation, then we get (again using Theo- rem 2.7.4)

$$\begin{aligned} L_{q0} &= (a \cup bb) \cdot L_{q0} \cup ba \cdot b^*a \cdot L_{q0} \cup b \\ &= (a \cup bb \cup bab^*a)L_{q0} \cup b. \end{aligned}$$

Again applying Lemma 2.8.2, this time with $L = L_{q0}$, $B \not\ni a \cup bb \quad bab^*a$ and $C = b$, gives

$L_{q0} = (a \cup bb \cup bab^*a)^* b.$

Thus, the regular expression that describes the language accepted by M is

$$(a \cup bb \cup bab^*a)^* b.$$

Completing the correctness of the conversion

It remains to prove that, for any DFA, the system of equations (2.2) and (2.3) can be solved. This will follow from the following (more general) lemma. (You should verify that the equations (2.2) and (2.3) are in the form as specified in this lemma.)

Lemma 2.8.3 Let $n \geq 1$ be an integer and, for $1 \leq i \leq n$ and $1 \leq j \leq n$, let B_{ij} and C_i be regular expressions such that $\varrho \notin B_{ij}$. Let $L_1, L_2, \ldots, L_n$ be languages that satisfy

$$L_i = \left(\bigcup_{j=1}^{n} B_{ij} L_j \right) \cup C_i \ \textit{for } 1 \leq i \leq n.$$

Then L_1 can be expressed as a regular expression only involving the regular expressions B_{ij} and C_i.

Proof. The proof is by induction on n. The base case is when $n = 1$. In this case, we have

$$L_1 = B_{11}L_1 \cup C_1.$$

Since $\varrho \notin B_{11}$, it follows from Lemma 2.8.2 that $L_1 = B_1{}^*{}_1C_1$. This proves the base case.

Let $n \geq 2$ and assume the lemma is true for $n-1$. We have

$$\begin{aligned} L_n &= \left(\bigcup_{j=1}^{n} B_{nj}L_j\right) \cup C_n \\ &= B_{nn}L_n \cup \left(\bigcup_{j=1}^{n-1} B_{nj}L_j\right) \cup C_n. \end{aligned}$$

Since $\varrho \notin B_{nn}$, it follows from Lemma 2.8.2 that

$$\begin{aligned} L_n &= B_{nn}^* \left(\left(\bigcup_{j=1}^{n-1} B_{nj}L_j\right) \cup C_n\right) \\ &= B_{nn}^* \left(\bigcup_{j=1}^{n-1} B_{nj}L_j\right) \cup B_{nn}^* C_n \\ &= \left(\bigcup_{j=1}^{n-1} B_{nn}^* B_{nj}L_j\right) \cup B_{nn}^* C_n \end{aligned}$$

By substituting this equation for L_n into the equations for L_i, $1 \leq i \leq n-1$, we obtain

$$\begin{aligned} L_i &= \left(\bigcup_{j=1}^{n} B_{ij}L_j\right) \cup C_i \\ &= B_{in}L_n \cup \left(\bigcup_{j=1}^{n-1} B_{ij}L_j\right) \cup C_i \\ &= \left(\bigcup_{j=1}^{n-1} \left(B_{in}B_{nn}^* B_{nj} \cup B_{ij}\right) L_j\right) \cup B_{in}B_{nn}^* C_n \cup C_i. \end{aligned}$$

Thus, we have obtained $n-1$ equations in $L_1, L_2, \ldots, L_{n-1}$. Since $\varrho \notin B_{in}B_{nn}^*B_{nj} \cup B_{ij}$, it follows from the induction hypothesis that L_1 can be expressed as a regular expression only involving the regular expressions B_{ij} and C_i.

Exercises

1.1 For each of the following languages, construct a DFA that accepts the language. In all cases, the alphabet is {0, 1}.

1. {w : the length of w is divisible by three}
2. {w : 110 is not a substring of w}
3. {w : w contains at least five 1s}
4. {w : w contains the substring 1011}
5. {w : w contains at least two 1s and at most two 0s}
6. {w : w contains an odd number of 1s or exactly two 0s}
7. {w : w begins with 1 and ends with 0}
8. {w : every odd position in w is 1}
9. {w : w has length at least 3 and its third symbol is 0}
10. {ϱ, 0}

1.2 For each of the following languages, construct an NFA, with the specified number of states, that accepts the language. In all cases, the alphabet is {0, 1}.

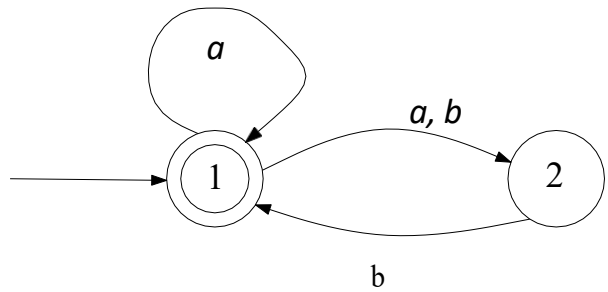
1. The language {w : w ends with 10} with three states.
2. The language {w : w contains the substring 1011} with five states.
3. The language {w : w contains an odd number of 1s or exactly two 0s} with six states.

1.3 For each of the following languages, construct an NFA that accepts the language. In all cases, the alphabet is {0, 1}.

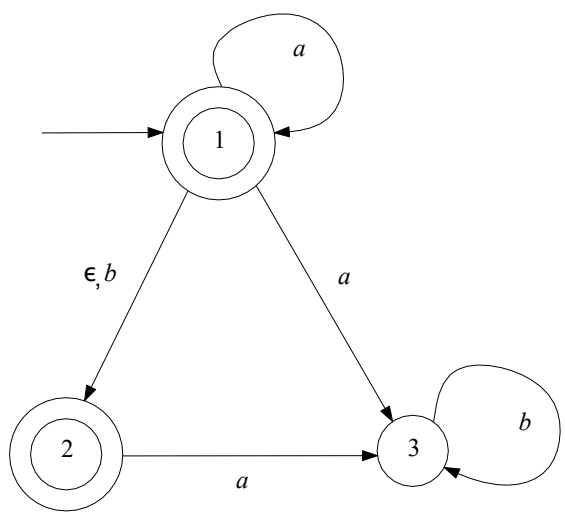
1. {w : w contains the substring 11001}
2. {w : w has length at least 2 and does not end with 10}

3. {w : w begins with 1 or ends with 0}

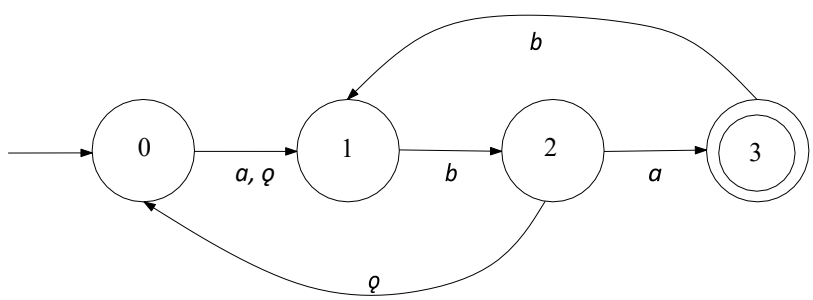
1.4 Convert the following NFA to an equivalent DFA.



1.5 Convert the following NFA to an equivalent DFA.



1.6 Convert the following NFA to an equivalent DFA.



1.7 For each of the following regular expressions, give two strings that are members and two strings that are not members of the language described by the expression. The alphabet is $\Sigma = \{a, b\}$.

1. $a(ba)^*b$.

2. $(a \cup b)^*a(a \cup b)^*b(a \cup b)^*a(a \cup b)^*$.

3. $(a \cup ba \cup bb)(a \cup b)^*$.

1.8 Give regular expressions describing the following languages. In all cases, the alphabet is {0, 1}.

1. {w : w contains at least three 1s}.

2. {w : w contains at least two 1s and at most one 0},

3. {w : w contains an even number of 0s and exactly two 1s}.

4. {w : w contains exactly two 0s and at least two 1s}.

5. {w : w contains an even number of 0s and each 0 is followed by at least one 1}.

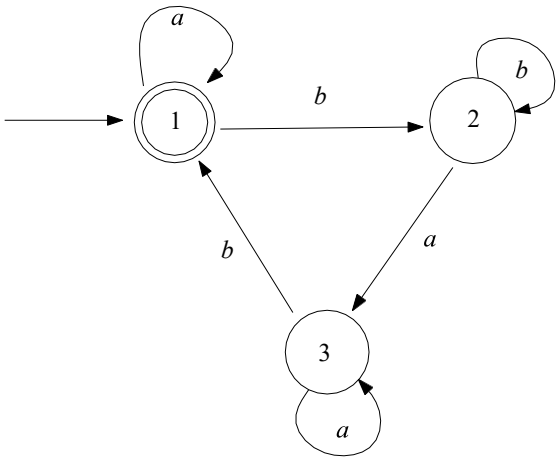
6. {w : every odd position in w is 1}.

1.9 Convert each of the following regular expressions to an NFA.

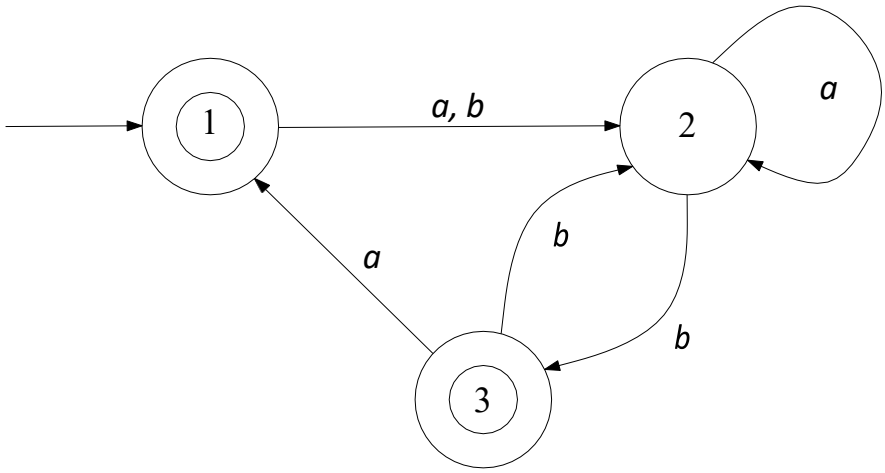
1. $(0 \cup 1)^*000(0 \cup 1)^*$

2. $(((10)^*(00)) \cup 10)^*$
3. $((0 \cup 1)(11)^* \cup 0)^*$

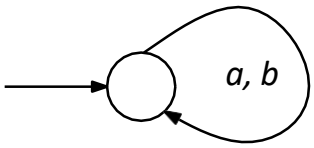
1.10 Convert the following DFA to a regular expression.



1.11 Convert the following DFA to a regular expression.



1.12 Convert the following DFA to a regular expression.



Chapter 3 Context-Free Languages

In this chapter, we introduce the class of context-free languages. As we will see, this class contains all regular languages, as well as some nonregular languages such as $\{0^n1^n : n \geq 0\}$.

The class of context-free languages consists of languages that have some sort of recursive structure. We will see two equivalent methods to obtain this class. We start with context-free grammars, which are used for defining the syntax of programming languages and their compilation. Then we introduce the notion of (nondeterministic) pushdown automata, and show that these automata have the same power as context-free grammars.

3.1 Context-free grammars

We start with an example. Consider the following five (substitution) rules:

$$S \rightarrow AB$$
$$A \rightarrow a$$
$$A \rightarrow aA$$
$$B \rightarrow bB$$
$$B \rightarrow bB$$

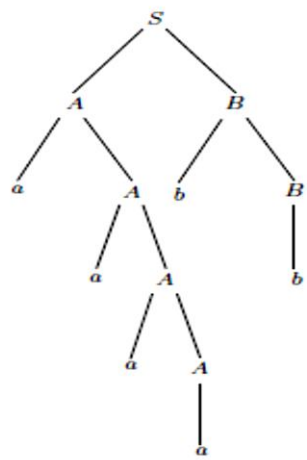
Here, S, A, and B are variables, S is the start variable, and a and b are terminals. We use these rules to derive strings consisting of terminals (i.e., elements of $\{a, b\}^*$), in the following manner:

1. Initialize the current string to be the string consisting of the start variable S.
2. Take any variable in the current string and take any rule that has this variable on the left-hand side. Then, in the current string, replace this variable by the right-hand side of the rule.
3. Repeat 2. until the current string only contains terminals.

For example, the string aaaabb can be derived in the following way:

$$
\begin{aligned}
S &\Rightarrow AB \\
&\Rightarrow aAB \\
&\Rightarrow aAbB \\
&\Rightarrow aaAbB \\
&\Rightarrow aaaAbB \\
&\Rightarrow aaaabB \\
&\Rightarrow aaaabb
\end{aligned}
$$

This derivation can also be represented using a parse tree, as in the figure below:



The five rules in this example constitute a context-free grammar. The language of this grammar is the set of all strings that

- can be derived from the start variable and
- only contain terminals.

For this example, the language is

$$\{a^m b^n : m \geq 1, n \geq 1\},$$

because every string of the form $a^m b^n$, for some m 1 and n 1, can be derived from the start variable, whereas no other string over the alphabet
{a, b} can be derived from the start variable.

Definition 3.1.1 A context-free grammar is a 4-tuple G = (V, Σ, R, S), where

1. V is a finite set, whose elements are called variables,

2. Σ is a finite set, whose elements are called terminals,

3. $V \cap \Sigma = \emptyset$,

4. S is an element of V ; it is called the start variable,

5. R is a finite set, whose elements are called rules. Each rule has the form $A \to w$, where $A \in V$ and $w \in (V \cup \Sigma)^*$.

In our example, we have $V = \{S, A, B\}$, $\Sigma = \{a, b\}$, and

$$R = \{S \to AB, A \to a, A \to aA, B \to b, B \to bB\}.$$

Definition 3.1.2 Let $G = (V, \Sigma, R, S)$ be a context-free grammar. Let A be an element in V and let u, v, and w be strings in $(V \cup \Sigma)^*$ such that $A \to w$ is a rule in R. We say that the string uwv can be derived in one step from the string uAv, and write this as

$$uAv \Rightarrow uwv.$$

In other words, by applying the rule $A \to w$ to the string uAv, we obtain the string uwv. In our example, we see that $aaAbb \Rightarrow aaaAbb$.

Definition 3.1.3 Let $G = (V, \Sigma, R, S)$ be a context-free grammar. Let u and v be strings in $(V \cup \Sigma)^*$. We say that v can be derived from u, and write this as $u \overset{*}{\Rightarrow} v$, if one of the following two conditions holds:

1. $u = v$ or
2. there exist an integer $k \geq 2$ and a sequence $u_1, u_2, \ldots, u_k$ of strings in $(V \cup \Sigma)^*$, such that

(a) $u = u_1$,

(b) $v = u_k$, and

(c) $u_1 \Rightarrow u_2 \Rightarrow \ldots \Rightarrow u_k$.

In other words, by starting with the string u and applying rules zero or more times, we obtain the string v. In our example, we see that $aaAbB \overset{*}{\Rightarrow} aaaabbbB$.

Definition 3.1.4 Let $G = (V, \Sigma, R, S)$ be a context-free grammar. The language of G

is defined to be the set of all strings in Σ^* that can be derived from the start variable S:

$L(G) = \{w \in \Sigma^* : S \overset{*}{\Rightarrow} w\}$.

Definition 3.1.5 A language L is called context-free, if there exists a context- free grammar G such that $L(G) = L$.

3.2 Examples of context-free grammars

1.2.1 Properly nested parentheses

Consider the context-free grammar $G = (V, \Sigma, R, S)$, where $V = \{S\}$, $\Sigma = \{a, b\}$, and

$$R = \{S \to \varrho, S \to aSb, S \to SS\}.$$

We write the three rules in R as

$$S \to \varrho | aSb | SS,$$

where you can think of "|" as being a short-hand for "or".

By applying the rules in R, starting with the start variable S, we obtain, for example,

$$\begin{aligned} S &\Rightarrow SS \\ &\Rightarrow aSbS \\ &\Rightarrow aSbSS \\ &\Rightarrow aSSbSS \\ &\Rightarrow aaSbSbSS \\ &\Rightarrow aabSbSS \\ &\Rightarrow aabbSS \\ &\Rightarrow aabbaSbS \\ &\Rightarrow aabbabS \\ &\Rightarrow aabbabaSb \\ &\Rightarrow aabbabab \end{aligned}$$

What is the language L(G) of this context-free grammar G? If we think of a as being a left-parenthesis "(", and of b as being a right-parenthesis ")", then L(G) is the language consisting of all strings of properly nested parentheses. Here is the explanation: Any string of properly nested parentheses is either

- empty (which we derive from S by the rule $S \rightarrow \varrho$),

  consists of a left-parenthesis, followed by an arbitrary string of properly nested parentheses, followed by a right-parenthesis (these are derived from S by first applying the rule $S \rightarrow aSb$), or

- consists of an arbitrary string of properly nested parentheses, followed by an arbitrary string of properly nested parentheses (these are derived from S by first applying the rule $S \rightarrow SS$).

1.2.2 A context-free grammar for a nonregular language

Consider the language $L_1 = \{0^n1^n : n \geq 0\}$. We have seen in Section 2.9.1 that L_1 is not a regular language. We claim that L_1 is a context-free language.
In order to prove this claim, we have to construct a context-free grammar G_1 such that $L(G_1) = L_1$.
Observe that any string in L_1 is either

- empty or
  - consists of a 0, followed by an arbitrary string in L_1, followed by a 1. This leads to the context-free grammar $G_1 = (V_1, \Sigma, R_1, S_1)$, where $V_1 = \{S_1\}$, $\Sigma = \{0, 1\}$, and R_1 consists of the rules

$$S_1 \rightarrow \varrho | 0S_11.$$

Hence, $R_1 = \{S_1 \rightarrow \varrho, S_1 \rightarrow 0S_11\}$.
To derive the string 0 1 from the start variable S_1, we do the following:

Starting with S_1, apply the $S_1 \rightarrow 0S_11$ rule exactly n times. This gives the string $0^nS_11^n$.

- Apply the rule $S_1 \rightarrow \varrho$. This gives the string 0^n1^n.

It is not difficult to see that these are the only strings that can be derived from the start variable S_1. Thus, $L(G_1) = L_1$.

In a symmetric way, we see that the context-free grammar $G_2 = (V_2, \Sigma, R_2, S_2)$,

where $V_2 = \{S_2\}$, $\Sigma = \{0, 1\}$, and R_2 consists of the rules

$$S_2 \rightarrow \varrho | 1S_20,$$

has the property that $L(G_2) = L_2$, where $L_2 = \{1^n0^n : n \geq 0\}$. Thus, L_2 is a context-free language.

Define $L = L_1 \cup L_2$, i.e.,

$$L = \{0^n1^n : n \geq 0\} \cup \{1^n0^n : n \geq 0\}.$$

The context-free grammar $G = (V, \Sigma, R, S)$, where $V = \{S, S_1, S_2\}$, $\Sigma = \{0, 1\}$, and R consists of the rules

$$\begin{array}{lcl} S & \rightarrow & S_1|S_2 \\ S_1 & \rightarrow & \epsilon|0S_11 \\ S_2 & \rightarrow & \epsilon|1S_20, \end{array}$$

has the property that $L(G) = L$. Hence, L is a context-free language.

1.2.3 A context-free grammar for the complement of a nonregular language

Let L be the (nonregular) language $L = \{0^n1^n : n \geq 0\}$. We want to prove that the complement $\overline{L}$ of L is a context-free language. Hence, we want to construct a context-free grammar G whose language is equal to $\overline{L}$. Observe that a binary string w is in $\overline{L}$ if and only if

1. $w = 0^m1^n$, for some integers m and n with $0 \leq m < n$, or
2. $w = 0^m1^n$, for some integers m and n with $0 \leq n < m$, or
3. w contains 10 as a substring.

Thus, we can write $\overline{L}$ as the union of the languages of all strings of type 1., type 2., and type 3.

Any string of type 1. is either

- the string 1,
- consists of a string of type 1., followed by one 1, or

- consists of one 0, followed by an arbitrary string of type 1., followed by one 1.

  Thus, using the rules

  $S_1 \rightarrow 1|S_1 1|0S_1 1$,

  we can derive, from S_1, all strings of type 1.

  Similarly, using the rules

  $S_2 \rightarrow 0|0S_2|0S_2 1$,

  we can derive, from S_2, all strings of type 2.

  Any string of type 3.

- consists of an arbitrary binary string, followed by the string 10, followed by an arbitrary binary string.

  Using the rules

  $X \rightarrow \varrho|0X|1X$,

  we can derive, from X, all binary strings. Thus, by combining these with the rule

  $$S_3 \rightarrow X10X,$$

  we can derive, from S_3, all strings of type 3.

  We arrive at the context-free grammar $G = (V, \Sigma, R, S)$, where $V = \{S, S_1, S_2, S_3, X\}$, $\Sigma = \{0, 1\}$, and R consists of the rules

$$\begin{array}{lcl} S & \rightarrow & S_1|S_2|S_3 \\ S_1 & \rightarrow & 1|S_1 1|0S_1 1 \\ S_2 & \rightarrow & 0|0S_2|0S_2 1 \\ S_3 & \rightarrow & X10X \\ X & \rightarrow & \epsilon|0X|1X \end{array}$$

To summarize, we have

$$S_1 \overset{*}{\Rightarrow} 0^m 1^n, \text{ for all integers } m \text{ and } n \text{ with } 0 \leq m < n,$$

$$S_2 \overset{*}{\Rightarrow} 0^m 1^n, \text{ for all integers } m \text{ and } n \text{ with } 0 \leq n < m,$$

$$X \overset{*}{\Rightarrow} u, \text{ for each string } u \text{ in } \{0, 1\}^*,$$

And

$$S_3 \overset{*}{\Rightarrow} w, \text{ for every binary string } w \text{ that contains 10 as a substring.}$$

From these observations, it follows that that L(G) = L.

Reference

Anil Maheshwari, Michiel Smid, Introduction to Theory of Computation, School of Computer Science, Carleton University, Ottawa, Canada, April 17, 2019